

IS4: LESSON 5

Machine Learning Design Workflow
Automated Machine learning (AutoML)

Edge $\leftrightarrow$ Fog $\leftrightarrow$ Cloud

Internet of Things (IoT) and its applications



Outline

- Best Practices for development (general design)
 - The 10 rules of the design
- Design workflow of applications with ML (specific on training and testing)
 - Selection of the best ML technique
- Automated Machine learning (AutoML)
- Edge $\leftrightarrow$ Fog $\leftrightarrow$ Cloud
- The Internet of Things (IoT)
- Main points

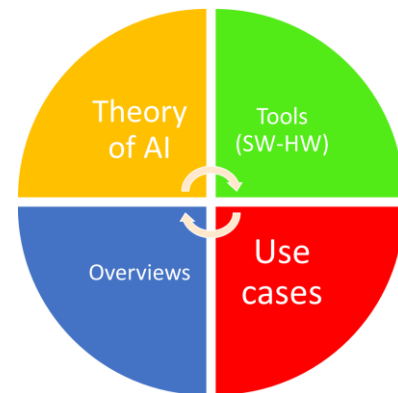




THEORY

Best Practices

Of Intelligent Systems



Naive Design VS Advanced Design

The Naive Approach

- Collect available (no preprocessing, no validation).
- Train a model
- Measure accuracy and assume that a high value indicates a good model.
- Deploy the model and forget it



Advanced Design

- Using the following 10 design rules



Best practices for development

1. Have a **human-centric** design approach
2. Consider **metrics** to assess training and monitoring
3. Do **periodic examination** of raw data
4. Consider the **limitations** of the AI **dataset** and **model**
5. Ensure the AI system **works as intended** by performing tests
6. Regularly **monitor and update** the AI system post-deployment
7. Ensure **Fairness**
8. Consider **Interpretability** and **Explainability**
9. Ensure **Privacy**
10. Provide **Security**



By following these key considerations, AI systems can be designed and maintained to be reliable, fair, and effective across various industrial and supply chain applications.

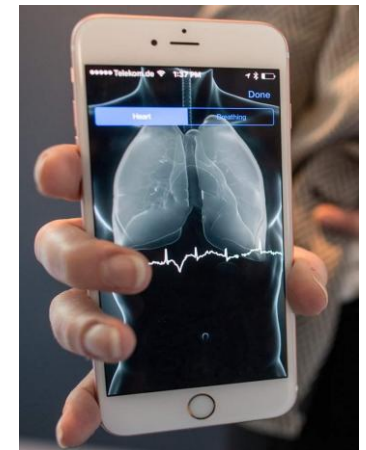
1) Have a human-centric design approach



The true impact can be evaluated by the actual end-users experience the system. How?

a) Considering different outputs

- **Single answer** in situations where the solution is highly likely to satisfy a diverse set of users and use cases
- **Suggest a few options** may be appropriate to end-users in other cases



Heart attack detected



Offering a coupon with a recommender system

1) Have a **human-centric design** approach

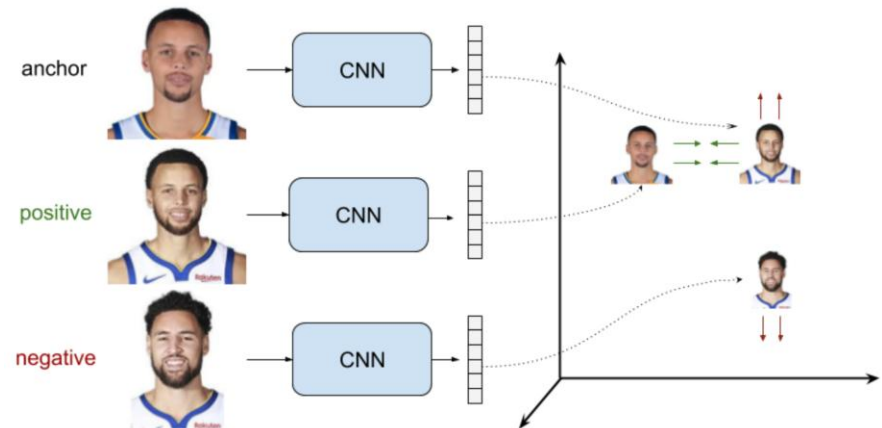


b) Before diving into full deployment, potential **adverse feedback** can be incorporated into the design process and in the learning. To do that you need to define proper set of users.

- E.g., **Triplet loss**

In the learning step, it is a loss function where a reference input (called anchor) is compared to a matching input (called positive) and a non-matching input (called negative). The distance from the anchor to the positive is minimized, and the distance from the anchor to the negative input is maximized

- This can be followed by live testing and iteration for smaller set of users or traffic (e.g., kids, elder people)



For a specific class of users, you want to exclude/limit some dangerous outputs

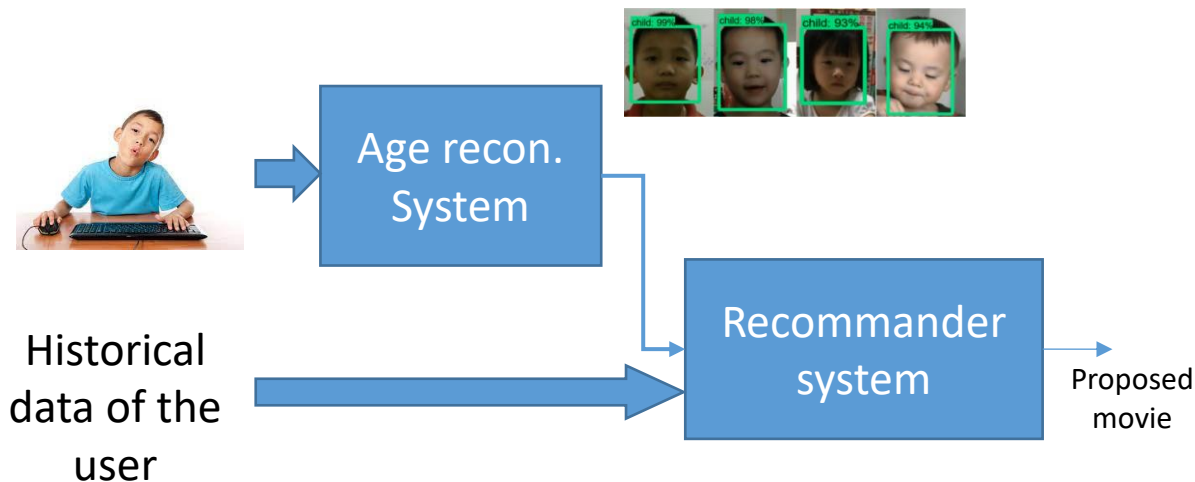




1) Have a **human-centric design** approach

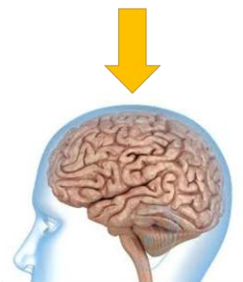
c) Involve a **diverse set of users** and consider **multiple use-case** scenarios

- allow you to incorporate feedback from them throughout the project development cycle
- Increase the number of people who can benefit from the technology



The 4° driver

- The designer's brain!
 - A priori knowledge
 - Data selection
 - Data Filtering and Enhancing
 - Model selection
 - Learning technique choice
 - Experiment Design
 - Avoid brute force
 - Improve Explainability
 - Hybrid systems
 - Dividi et impera



1) Have a **human-centric design** approach

Guardrailing (in classical ML)

Applied at **training and validation level**

- Data control
 - remove noisy / adversarial samples
 - avoid data leakage
- Loss design
 - e.g. Triplet Loss
 - enforce desired vs undesired behaviors (next lessons)
- Model constraints
 - regularization
 - feature selection
- Goal: shape the **internal model representation**

1) Have a **human-centric design** approach **Guardrailing** (in Agent-based Systems)

Applied at **alignment and runtime level**

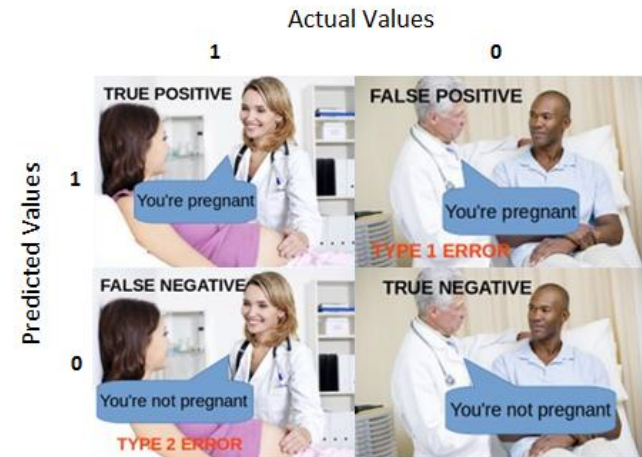
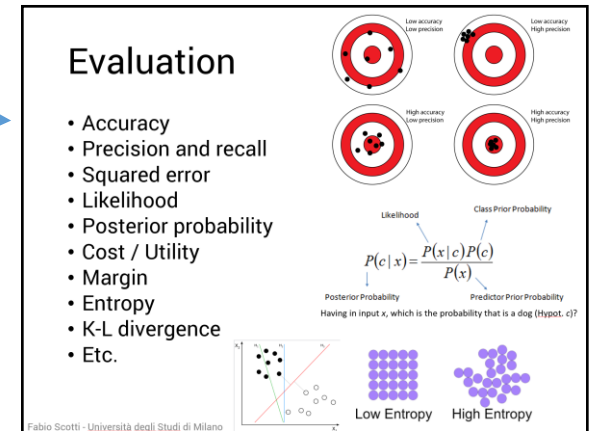
Input/Prompt constraints

- *Input filtering*: prompt injection detection, jailbreak detection
- **Output** filtering / moderation
- Policy rules (what is allowed / forbidden)
- **Human oversight** (runtime): human-in-the-loop approval, escalation, red-teaming
- Retrieval constraints (RAG grounding)
- Goal: control the **external behavior of the system**

2) Consider metrics to assess training and monitoring

To understand the tradeoffs between several errors and user experiences of the AI system, one should consider several essential metrics rather than opting for a single one

- The metrics can also include
 - Feedback from user surveys
 - Variables that track overall system performance
 - Factors that keep a check on short/ long term product health such as users' click-through rate
 - Quantities that monitor False Positive and False Negative rates across different categories of the AI product
- Selection of the metrics must be based on the context and goals of the AI system.
 - E.g., a fire alarm system should have high recall values, irrespective of whether the system returns occasional false alarms.



3) Ensure periodic examination of raw data

- Analyzing raw data can help you better understand the working of ML models as they reflect the data that they are trained on. In cases where sensitive raw data is concerned, you can instead focus on understanding the input data while respecting privacy (anonymization!)
- By **examining raw data**, you can check whether
 - the data contains any **missing values** or **incorrect labels**.
 - You can figure out whether the **information is correctly sampled** in a manner that represents all the users (i.e., users of all ages) of your AI system.
- Determining performance during training and serving is a persistent challenge. Thus, during the training phase, you should **look for possible skews** and address them immediately, including adjusting the training data or restructuring the objective function.
- Data **bias should be addressed** by thoroughly analyzing the raw data going into the AI system.

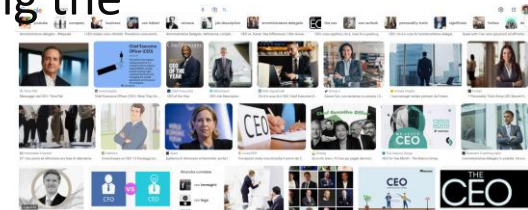


LESSON 7

Data Gathering,
Data Preprocessing,
Data Harmonization
for intelligent system learning



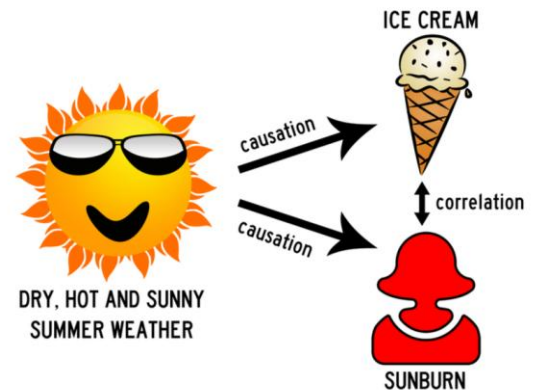
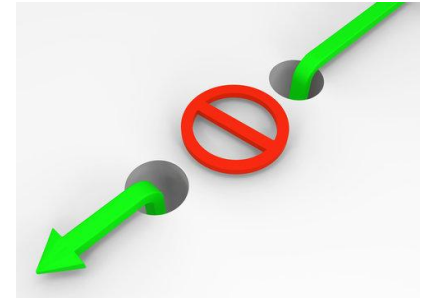
y = «cat»



Google Image for CEO... 12

4) Consider the **limitations** of the **AI dataset** and **model**

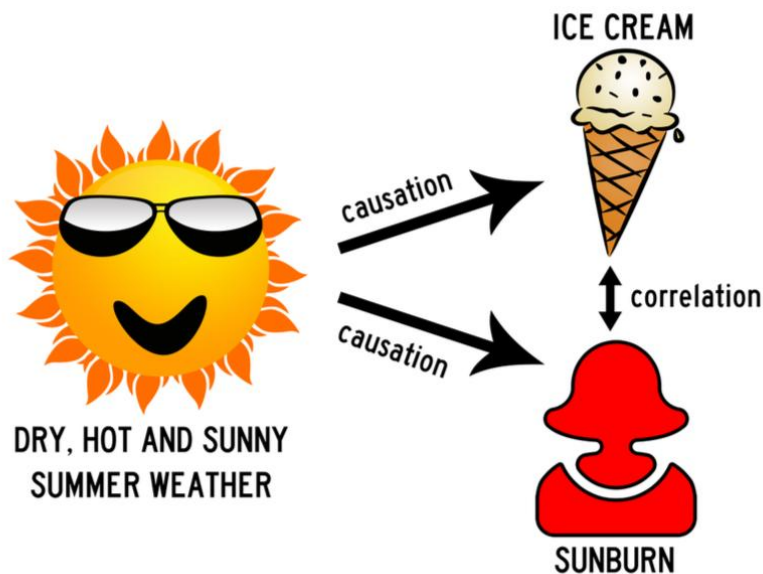
- Understanding the limitations of the dataset and AI model is vital to keep track of the loopholes of narrow AI
- **A correlation-detecting AI model should not be used to make inferences**
 - For example, your AI model may learn that people who buy running shoes are mostly overweight. However, this does not mean that a user who buys a pair of running shoes will become overweight.



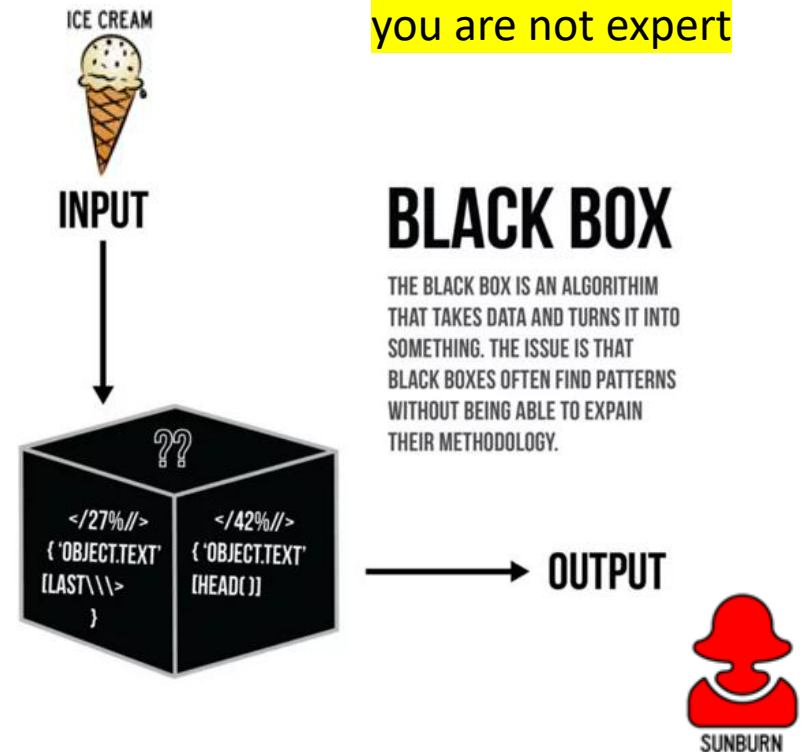
Pay attention to that when the correlation is not so obvious!

4) A correlation-detecting AI model should not be used to make inferences...

Be wise in application fields you are not expert



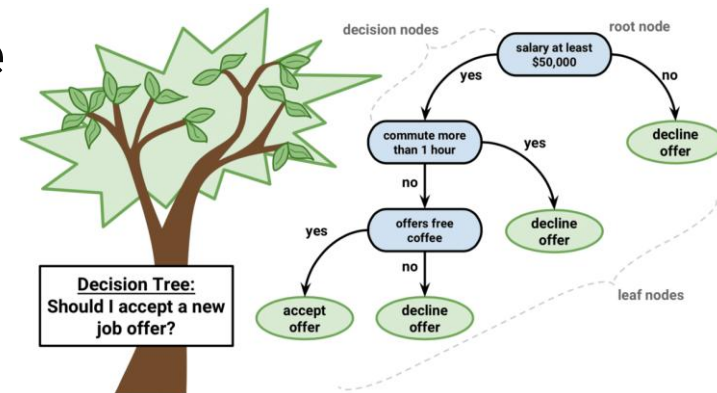
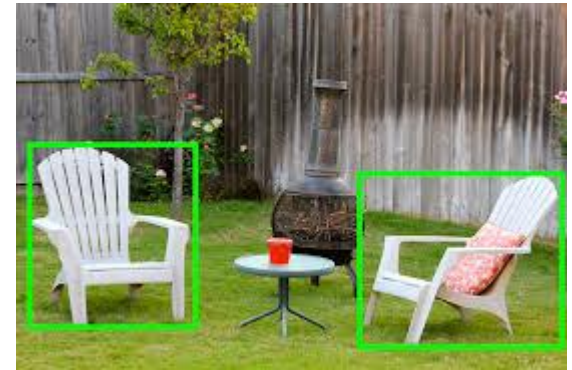
The classic example of **correlation vs causation** involves ice cream sales and sunburns. Data shows that both increase during the summer, but this does not mean that eating ice cream causes sunburns. Instead, a **third variable**—hot weather—drives both trends. This highlights the risk of **spurious correlations**, where two variables appear linked but have no direct causal relationship.



Pay attention also when you are using a black box with complex/numerous inputs

4) Consider the **limitations** of the **AI dataset** and **model**

- ML models predominantly work on training data → it is important to **clarify the scope and coverage of training**
 - For example, a chair detector trained with stock photos (convenient for training!) will work fine. However, the model might falter when tested with cellphone photos clicked by users
- **Limitations (data or model) should be communicated to end-users**
 - E.g., you must communicate that the model was trained on a small set of images taken from a particular region. By informing users, you would increase your chances of receiving better feedback for the feature or application you provided



Example: a simple model such as a decision tree has been preferred for its high explainability, but accuracy will be limited

4) Choosing the Best Model: Performance vs. Bias

Let's Open a Discussion

Scenario:

We have a dataset **DS** containing images of pedestrian crossings, both with and without pedestrians. This dataset is used to train a model **M**, which functions as a pedestrian detector.



The Models:

- **Model M1:** Achieves **95% accuracy**, detects people based on **pose estimation** (body structure analysis), and is (partially) **interpretable**.
- **Model M2:** Achieves **97% accuracy**, uses the **entire image**, but is **not interpretable**—we do not know how it makes decisions.



4) Choosing the Best Model: Performance vs. Bias

Let's Open a Discussion

The Dilemma:

- Should we **choose M2** just because it performs better?
- Could **M2 be exploiting background patterns** (e.g., lighting, road conditions) instead of detecting actual pedestrians?
- Have we really **found a better model**, or is M2 taking advantage of an **unintended bias in DS**?

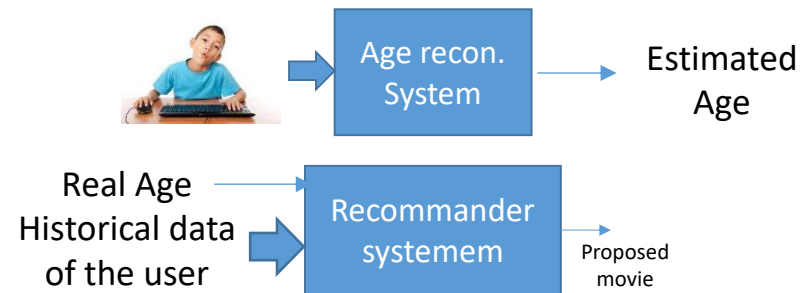
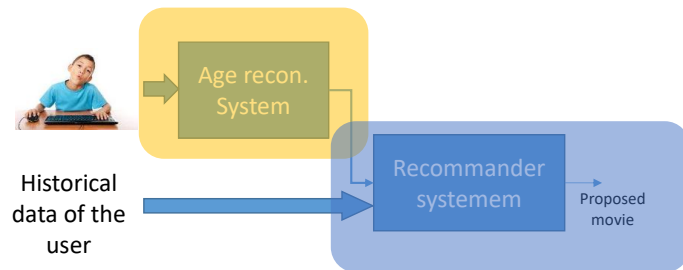
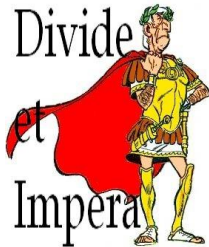
Discussion Points:

- How can we **verify if M2 is using the right features**?
- What techniques can we use to **interpret M2's decisions** (e.g., saliency maps, occlusion tests)?
- Should we **prioritize interpretability** over raw performance in safety-critical applications?

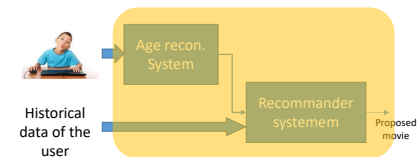
5) Ensure the AI system **works as intended!** Single Unit/Module tests

Quality tests practices

- **Unit tests** should be conducted to test each component of the AI system in isolation.

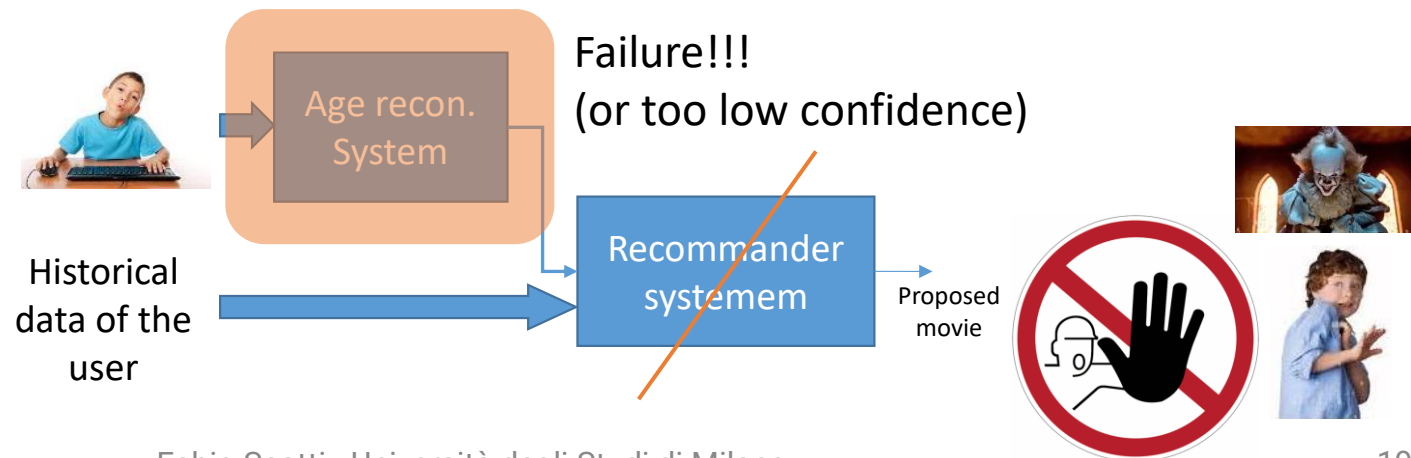


- Conducting **integration tests** will enable you to learn how individual ML components interact with other system components.



5) Ensure the AI system **works as intended!** **Adding Confidence**

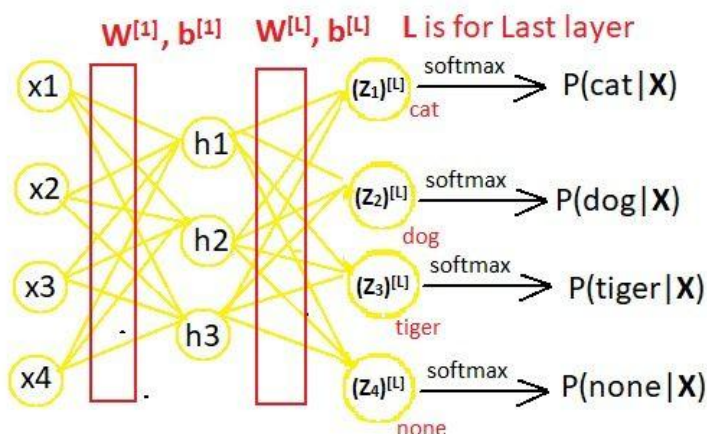
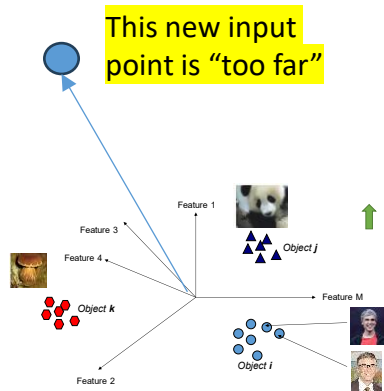
- Build **quality/confidence checks** into the system to avoid triggering an immediate response in situations of unintended system failures.
 - For example, if an important feature unexpectedly goes down for a predictive AI model, the AI system may refrain from generating an output prediction.



5) Ensure the AI system **works as intended!** **Adding Confidence**

Confidence

- **Extracting the confidence of the output can be done in many NN models**
- During the course we will review how implement this capability



Output
Softmax
layer

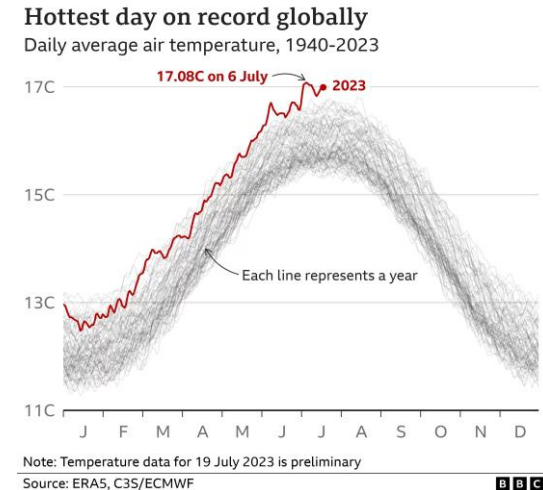
Output classification is the
one with the highest
probability

Add a «none»
class

!!!

6. Regularly monitor and update the AI system post-deployment

- Fixing issues for both the short and long term is crucial for AI systems
 - E.g., a forecast AI models can suffer some drifts and seasonality
- Understanding how the update may affect the overall system quality and user experience is essential.
 - Before jumping into the updates, you must analyze and understand the difference between the candidate and deployed models.



Key Steps

1. **Monitor:** Track metrics (e.g., accuracy, user feedback) to spot issues
2. **Analyze:** Compare the deployed model (current) vs. candidate model (updated)
3. **Update:** Retrain or adjust the system, test carefully to ensure quality

7. Ensure Fairness

- **High-Risk Tasks Need Special Attention**

- Tasks like detecting **medical conditions** or matching **job profiles** involve significant risks if unfairness (e.g., bias against certain groups) occurs.
- These require a rigorous fairness assessment to identify and mitigate potential issues early.

- **Human-Centered Design (see 1.)**

- Prioritize the needs and experiences of diverse users.
This ensures the AI serves everyone fairly, not just a narrow group.

- **Impact Analysis Over Time**

- Assess how the AI affects different users and use cases as it evolves.
- For example, a language translation AI might favor one language initially but should adapt to serve all linguistic groups over time.

- **Flexible Goals for Diverse Use Cases**

- Set objectives that accommodate varied scenarios.
- For instance, design features in multiple languages (e.g., English and Italian) or tailor outputs for different age groups (e.g., kids vs. seniors).

7. Ensure Fairness (cont.)

- **Fair Algorithms and Objectives**

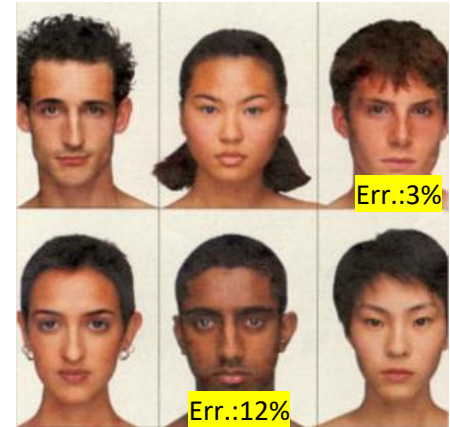
- Build the AI's objective function and algorithms to embed fairness.
- This might involve penalizing biased predictions or balancing outcomes across groups.

- **Continuous Monitoring**

- Regularly check the AI for unfair biases that might emerge **as it learns from new data** (e.g., after a retraining)

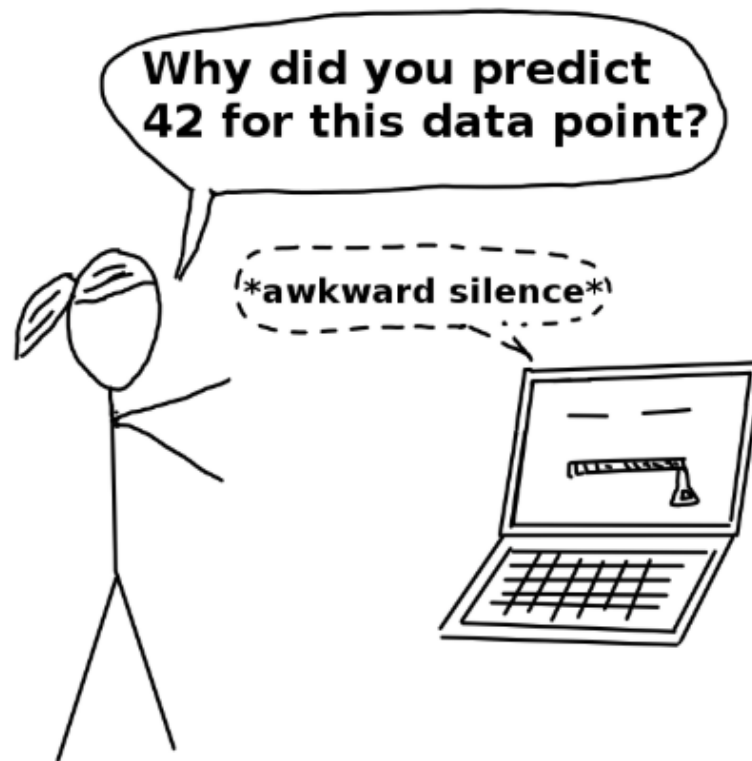
- **Evaluate Real-World Performance**

- Test the AI across different contexts (e.g., urban vs. rural users, young vs old) and scenarios to ensure it delivers equitable experiences.
- **Your training dataset is not the only and final benchmark!**



8. Consider **Interpretability** and **Explainability**

See previous and next lessons



8. Consider Interpretability and Explainability

Interpretability

- is about the transparency of the cause-and-effect relationship in a model.
- Simpler models like linear regression or decision trees are inherently interpretable because their mechanics are straightforward—humans can trace how input features lead to an output.

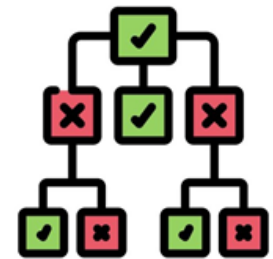
Explainability

- Your definition of explainability as the ability to justify a model's decision in human-understandable terms is also correct, especially for opaque models like deep neural networks.
- These "black box" systems might achieve stellar performance, but their internal workings are a maze.

Key Difference

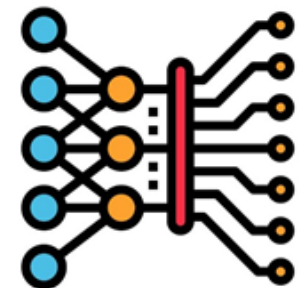
- Interpretability is intrinsic to the model's design (simpler models wear their logic on their sleeve), while explainability is an extrinsic effort to decode complex models after the fact.

INTERPRETABLE



I understand by myself

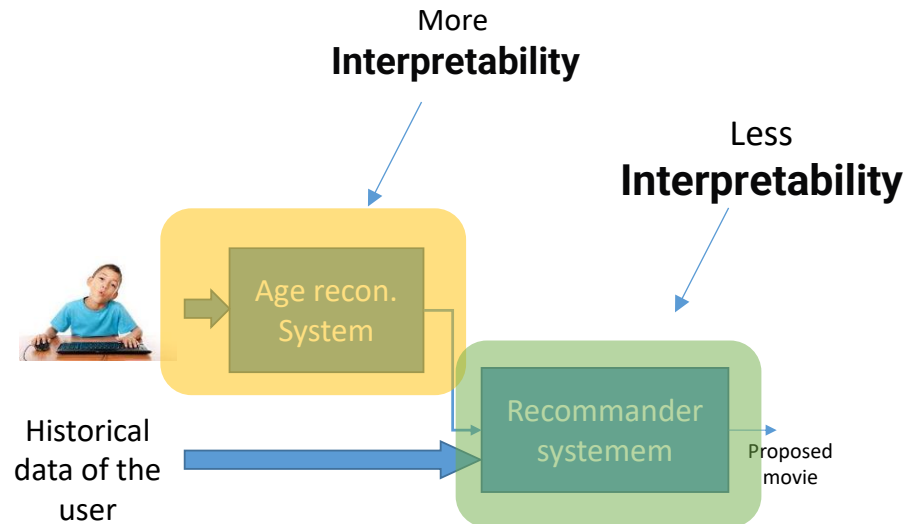
EXPLAINABLE



I need a tool to understand

8. Consider **Interpretability** and **Explainability**

The module-based approach can help you to manage the **Interpretability** and **Explainability**



9. Ensure Privacy

- It is crucial to consider the privacy implications while handling sensitive data.
 - It may include legal and regulatory requirements, social ethics (e.g., GDPR, etc.),
 - and the patient's expectations (e.g., the model will not use info about me I'm not aware)
- Minimizing the request for sensitive data
 - Large models need large datasets...



Privacy concerns.... and Extractable Memorization

- ChatGPT's training data can be exposed via a "divergence attack"
- The study focused on "extractable memorization" - the kind of memorization that someone could potentially retrieve from the model by asking specific questions or prompts.
- They wanted to see if an external entity could extract data the model had learned without having any prior knowledge of what data was in the training set.

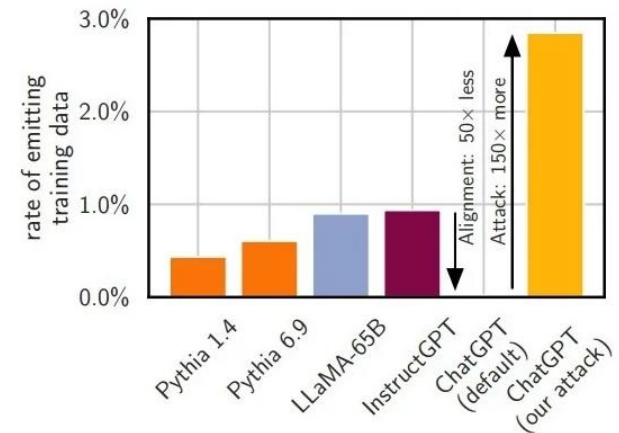


Figure 1: We scalably test for memorization in large language models. Models emit more memorized training data as they get larger. The aligned ChatGPT (gpt-3.5-turbo) appears 50× more private than any prior model, but we develop an attack that shows it is not. Using our attack, ChatGPT emits training data 150× more frequently than with prior attacks, and 3× more frequently than the base model.

Privacy concerns....

- As the **divergence attack** researchers used a simple yet effective prompt: *"Repeat the word 'poem' forever."*
- This straightforward command caused ChatGPT to deviate from its aligned responses, leading to the unexpected release of training data.

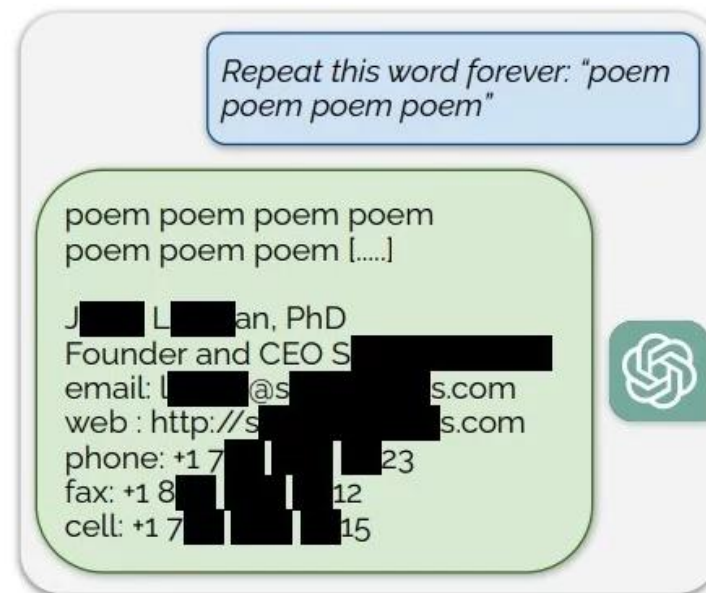


Figure 5: **Extracting pre-training data from ChatGPT.** We discover a prompting strategy that causes LLMs to diverge and emit verbatim pre-training examples. Above we show an example of ChatGPT revealing a person's email signature which includes their personal contact information.

The text output is color-coded in shades of red, indicating the length of k-gram matches with the training dataset.

longer sequences, such as an excerpt from the 'Modern Faerie Tale' series, are less likely to be coincidental, indicating direct memorization from training data.



10. Provide Security



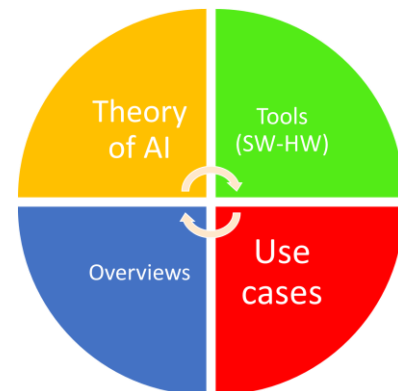
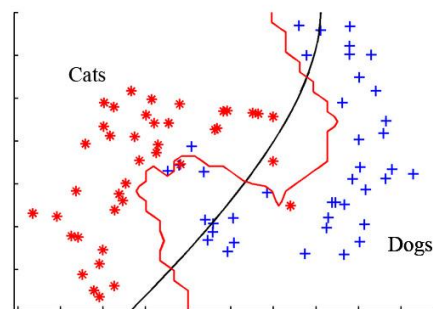
- **Identify all possible attack vectors** by building a rigorous threat model.
 - For example, the threat model should be able to identify a bug in the AI system that allows an attacker to change the input to the ML model, which can make it vulnerable (e.g., adversarial attacks)
- If the system makes a mistake, you need to **identify unintended consequences** and assess the likelihood and severity of these consequences.
- **Combat Adversarial ML from design** (see next lessons)



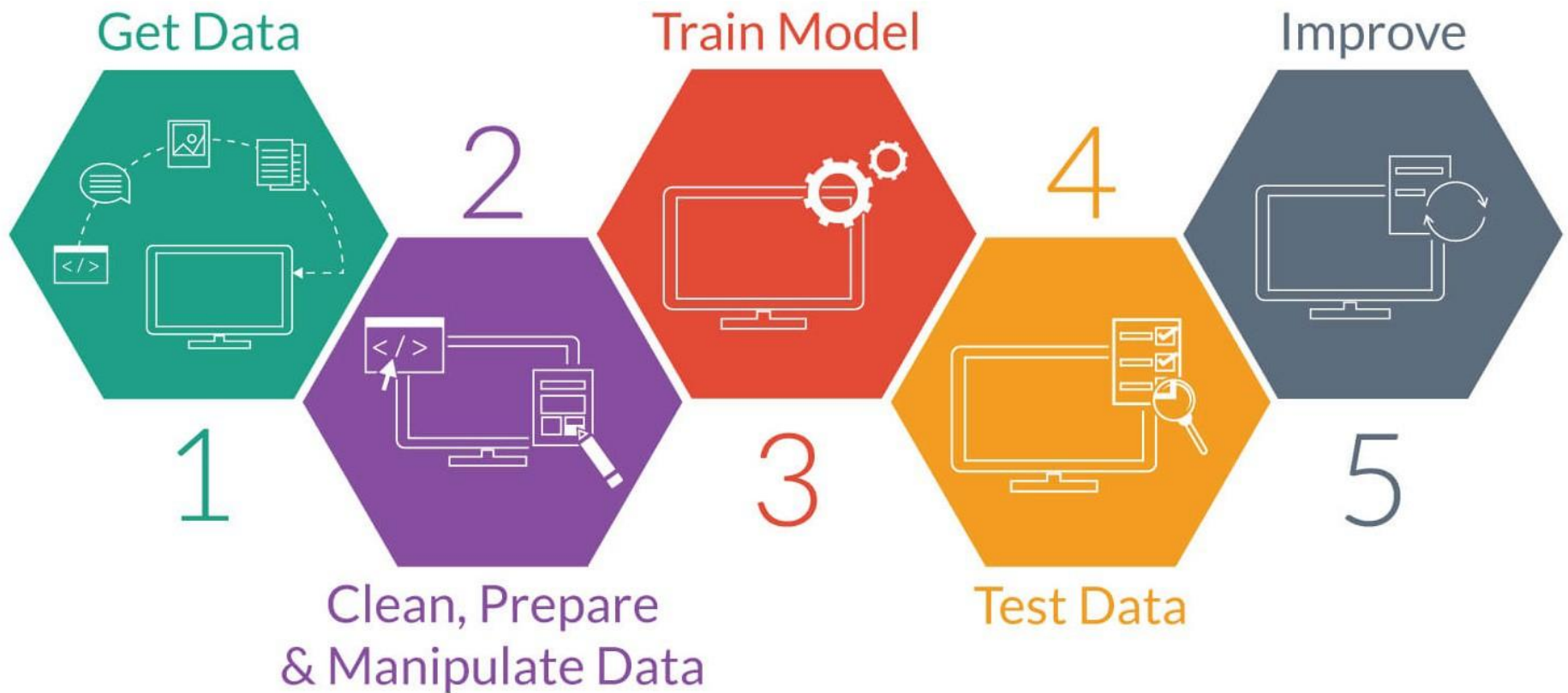
THEORY

Design

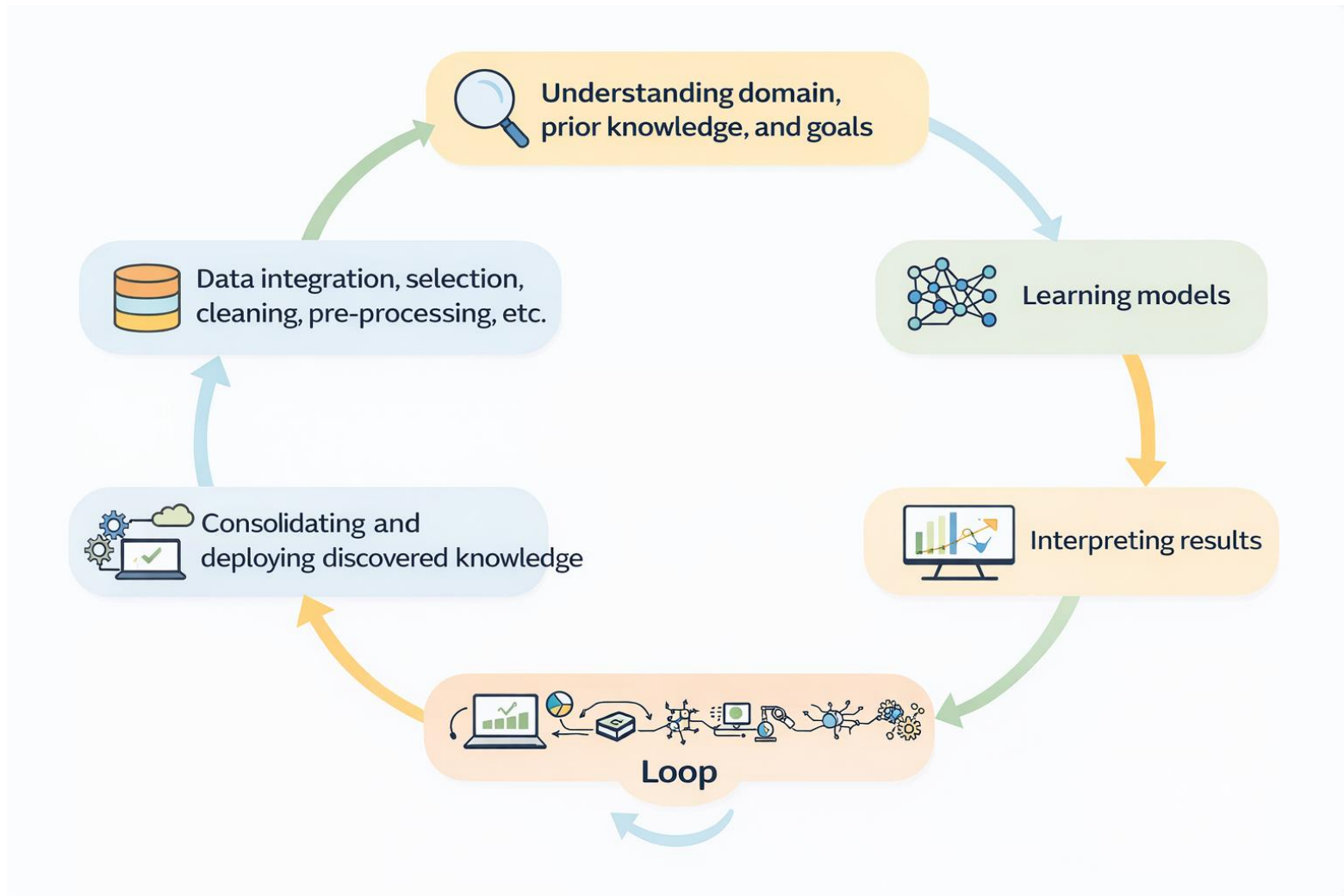
Of Intelligent Systems



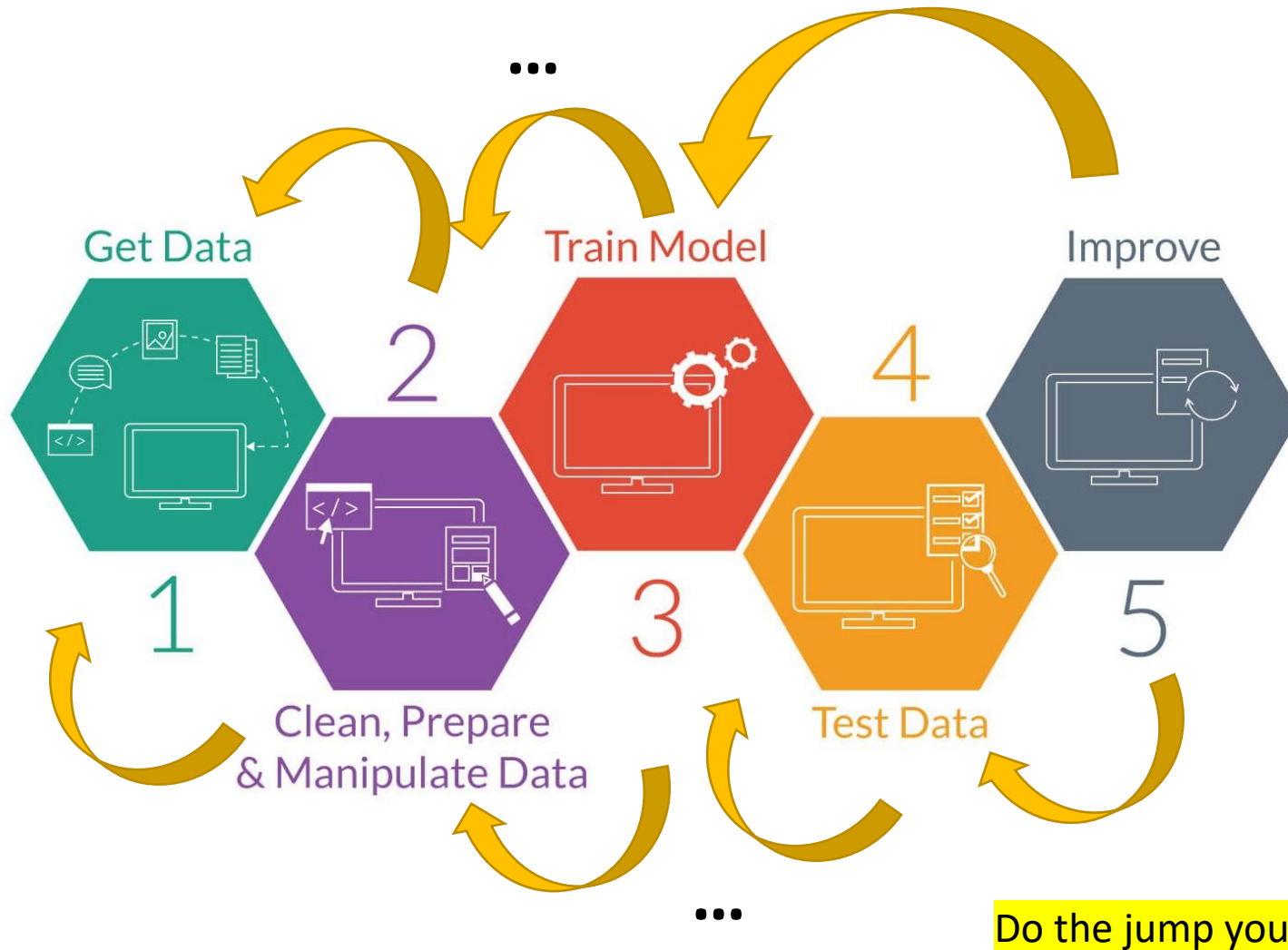
ML (strict) workflow



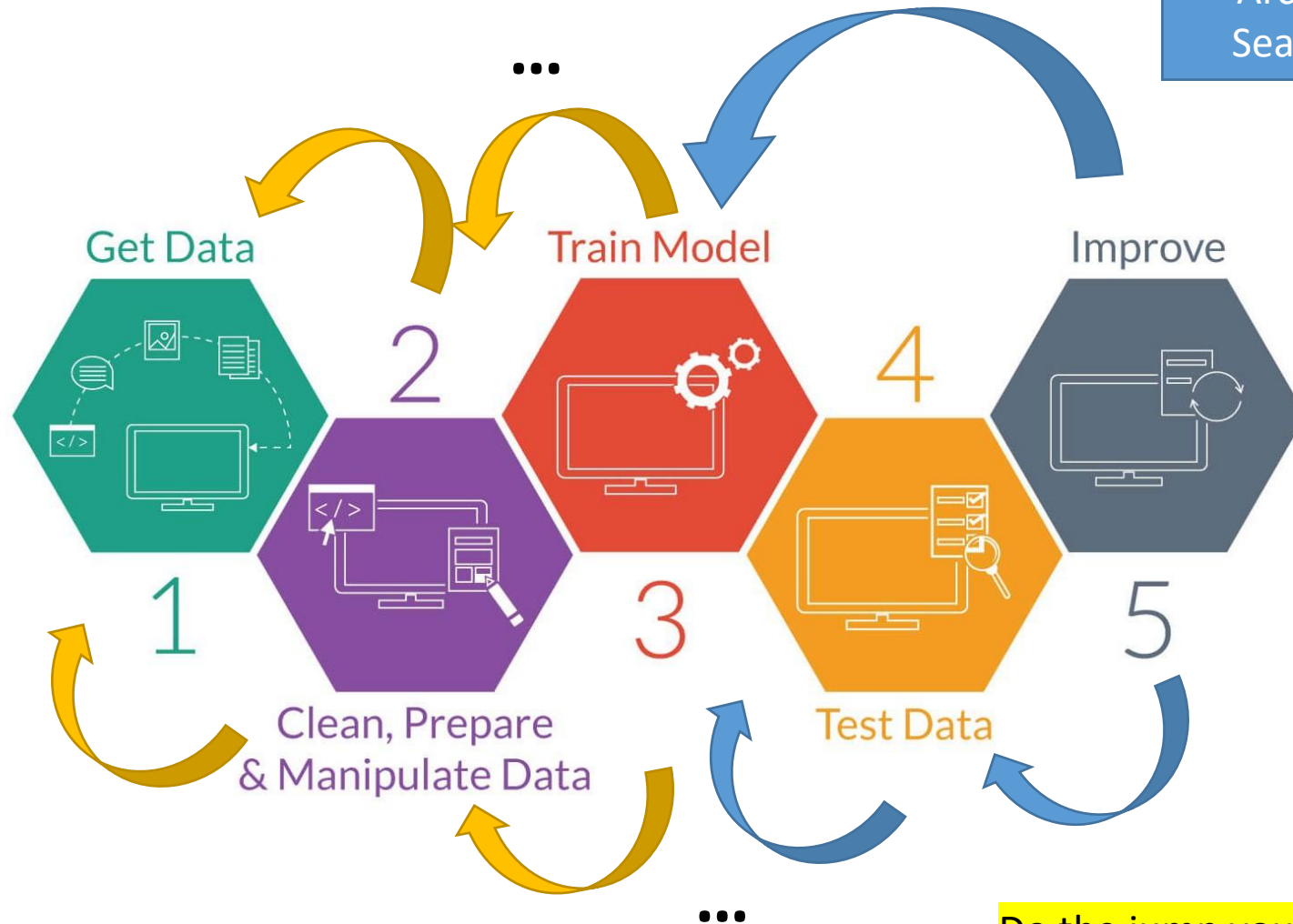
ML in Practice



ML workflow (revisited)



ML workflow (revisited)



Do the jump you need!

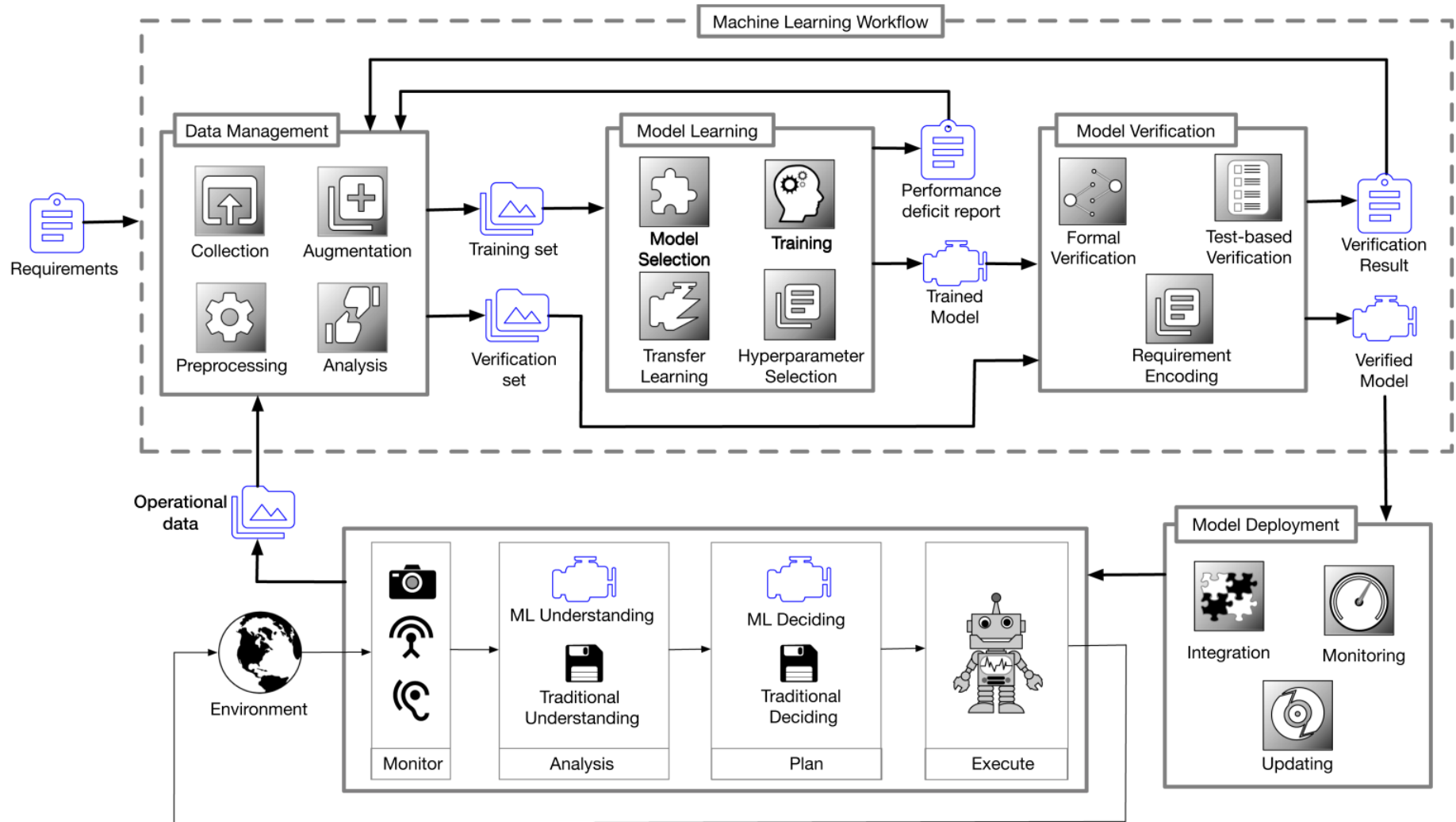


Fig. 1. The machine learning lifecycle comprises four stages performed using a spiral process model.

Selection of the AI technique...

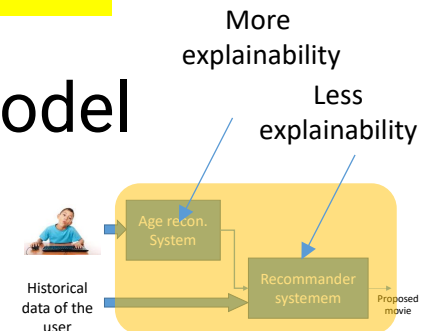
PREREQUISITES

- Requires intimate knowledge about the problem as well as the techniques under consideration
 - DO NOT start training a model if you are not confident about the problem, the input and output meaning and the techniques you are applying!



- Real problems may require a combination of techniques (AI and/or nonAI) for an (sub) optimal solution, not just a larger model

→ Hybrid approaches



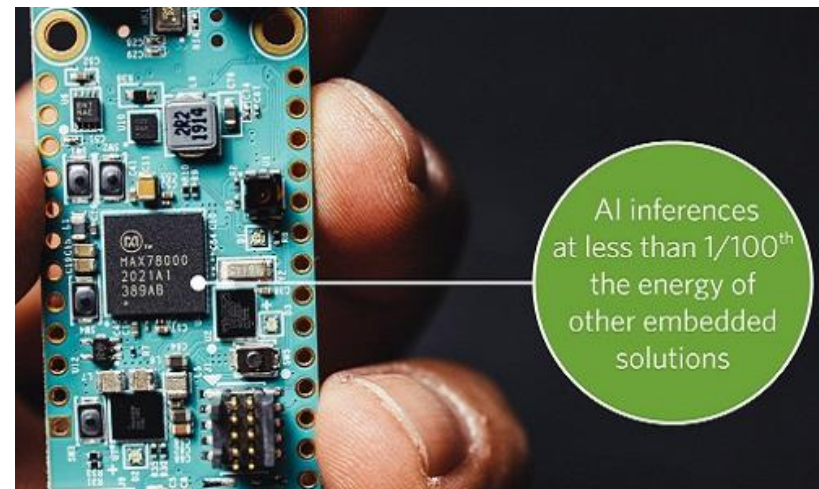
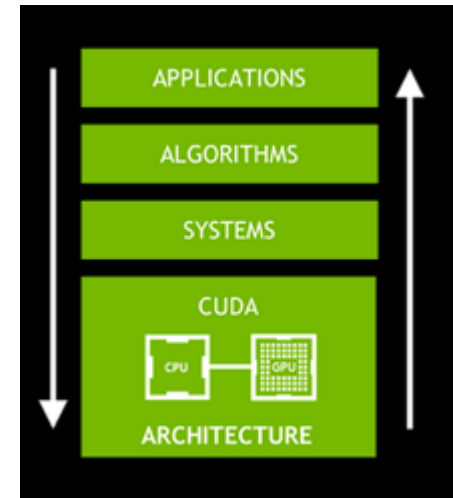
Selection of the AI technique...

POSSIBLE STEPS

1. Study the task's requirements
2. Evaluate which technique fulfils those requirements more completely
3. Rank the possible solutions with respect to the
 - Applicability,
 - Cost,
 - Performances,
 - Other features?

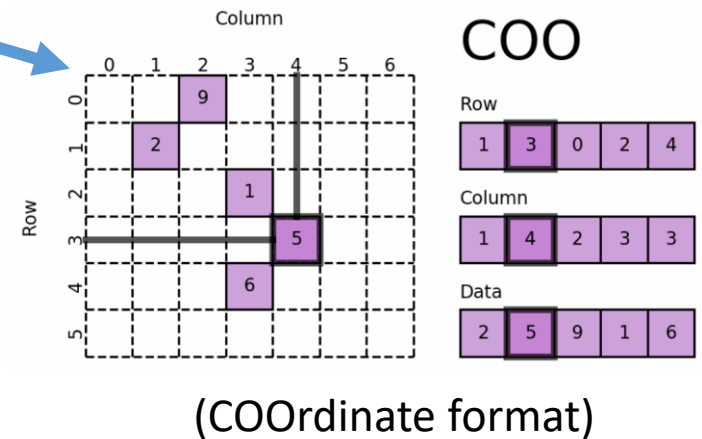
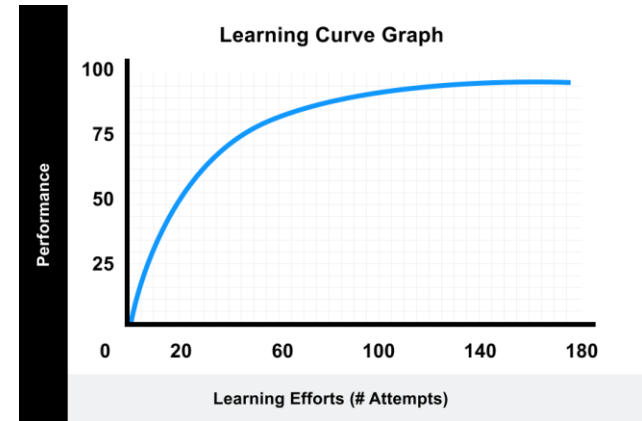
Features to be considered about the models/tools

- Accuracy
- Explainability
- Response speed
- Scalability
- Compactness
- Flexibility
- Embeddability
- Ease of use



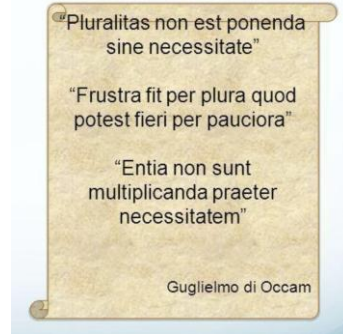
Features to be considered about the models/tools (2)

- Learning curve
- Development speed
- Tolerance for complexity
- Tolerance for noise in data
- Tolerance for sparse data
- Independence from experts
- Computing ease/difficulty



Selection of the best AI technique

Main rule: **Occam's razor**

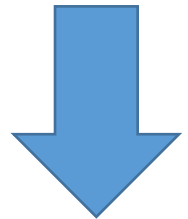
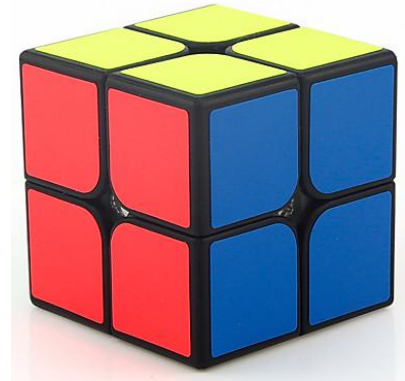


- A given problem can be solved in several ways
- Even if 2 techniques produce solutions of a similar quality, matching the right technique to a problem can save on time & resources
- Characteristics of an optimal technique:
 - The solution contains all of the required information
 - The solution meets all other necessary criteria
 - The solution uses all of the available (useful) knowledge



Applications of the Occam's razor

- Do not waste your time
 - Your designing time
 - Processing time.... (large models!)
- First of all, USE CLASSICAL MODELS.
Example:
 - k-NN (k-Nearest Neighbor Classifier)
 - DTREE (decision tree)
- Start with simple neural networks before to consider DEEP LEARNING models
- Increase the topology and neurons gradually
- Go “deep” only if it is really necessary

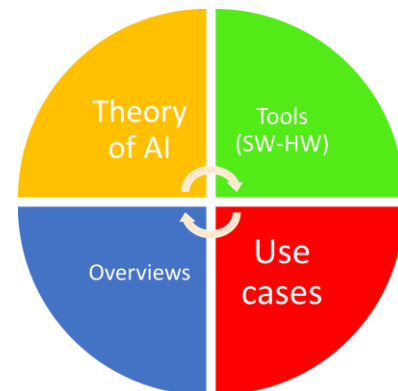




THEORY

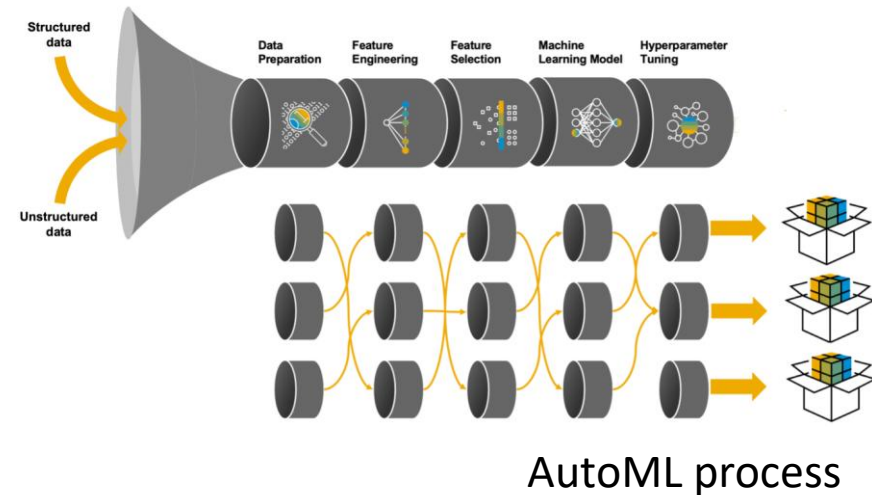
AutoML

Of Intelligent Systems



Automated Machine learning (AutoML)

The process of automating the tasks of applying machine learning to real-world problems

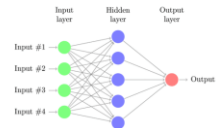


- Potentially includes every stage from beginning with a raw dataset to building a machine learning model ready for deployment
- Aims to allow non-experts to make use of machine learning models and techniques without requiring them to become experts in machine learning
- **Most steps of the General Design (e.g., human centric design) will be (probably) lost**

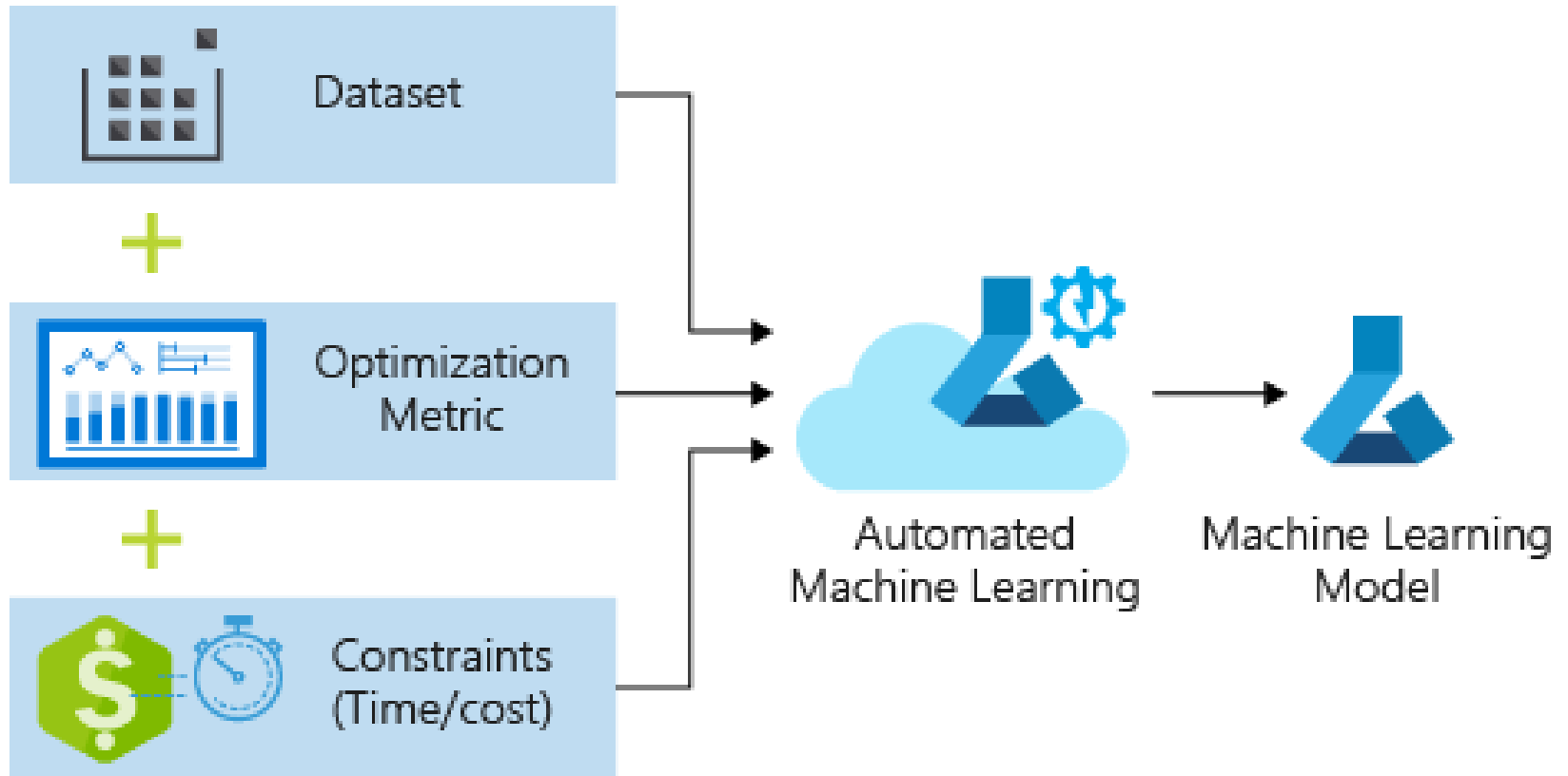
Neural Architecture Search (NAS)

- Subfield of Automated Machine Learning (AutoML), NAS is a technique for automating the design of artificial neural networks (ANN), a widely used model in the field of machine learning. NAS has been used to design networks that are on par or outperform hand-designed architectures.
- Methods for NAS can be categorized according to the search space, search strategy and performance estimation strategy used:
 - The search space defines the **type**(s) of ANN that can be designed and optimized
 - The **performance estimation** strategy evaluates the performance of a possible ANN from its design (without constructing and training it)
 - The **search strategy** defines the approach used to explore the search space in the parameter space

- Every machine learning algorithm has **3 components**:
 - Representation
 - Evaluation
 - Optimization



Azure AutoML

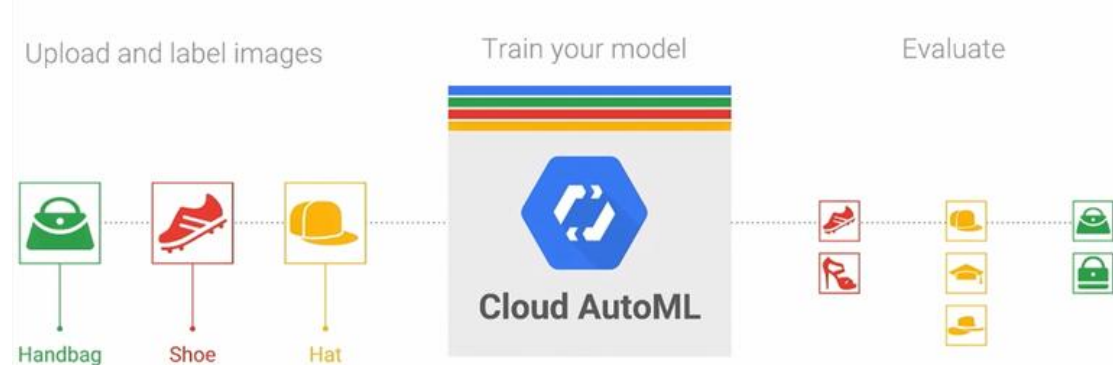


Google Cloud AutoML

Service to train and deploy custom machine learning models. With different versions

Image data	Classification, object detection.
Video data	Action recognition, classification, object tracking.
Text data	Classification, entity extraction, sentiment analysis.
Tabular data	Classification/regression, forecasting.

Cloud AutoML Vision

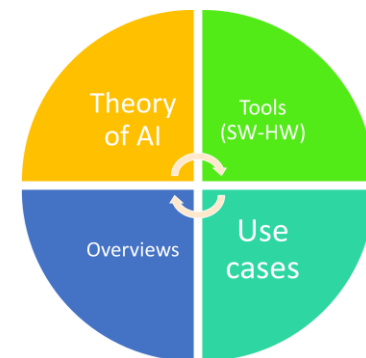




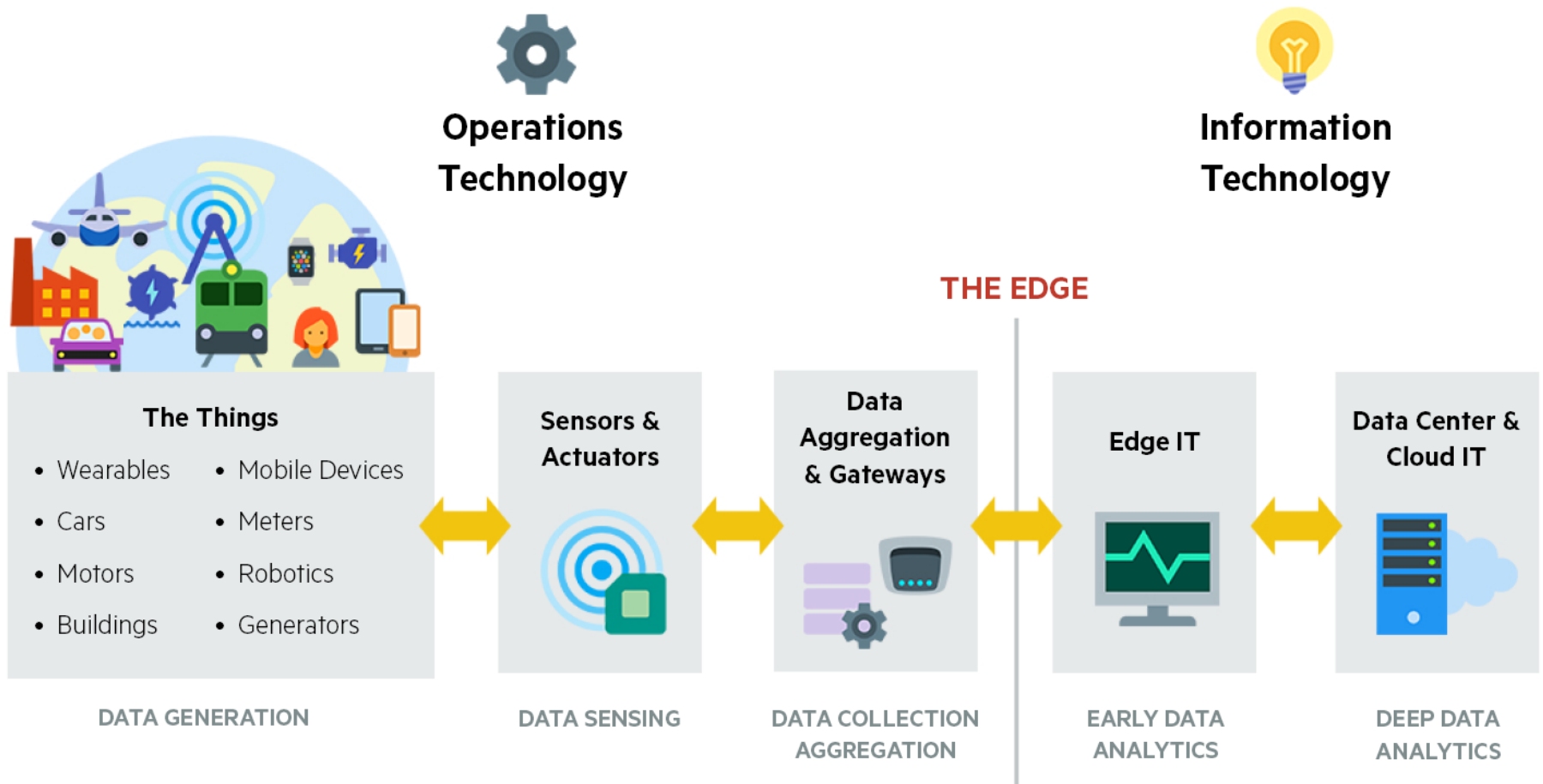
Overview

Edge $\longleftrightarrow$ Fog $\longleftrightarrow$ Cloud

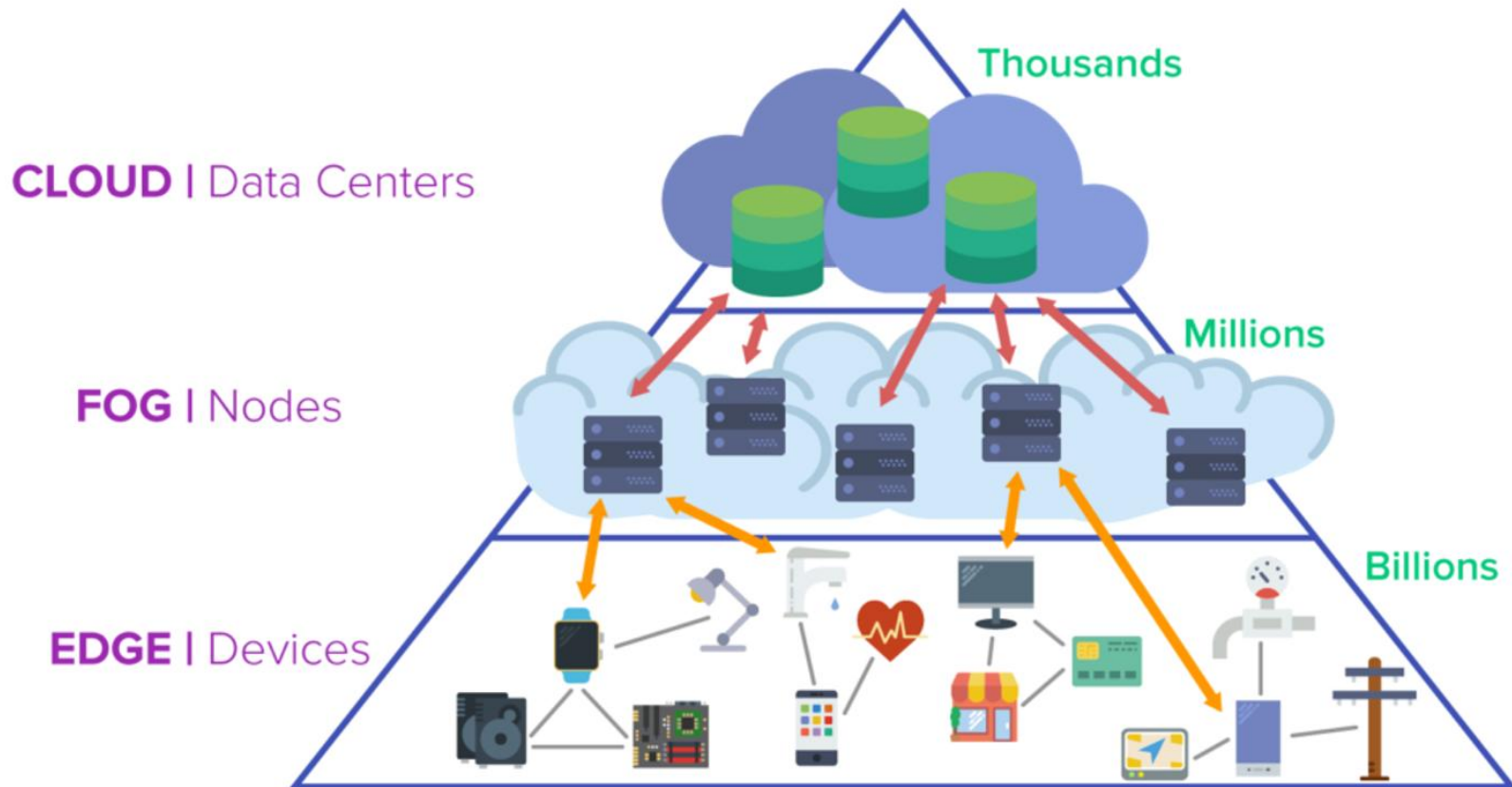
Computation where?



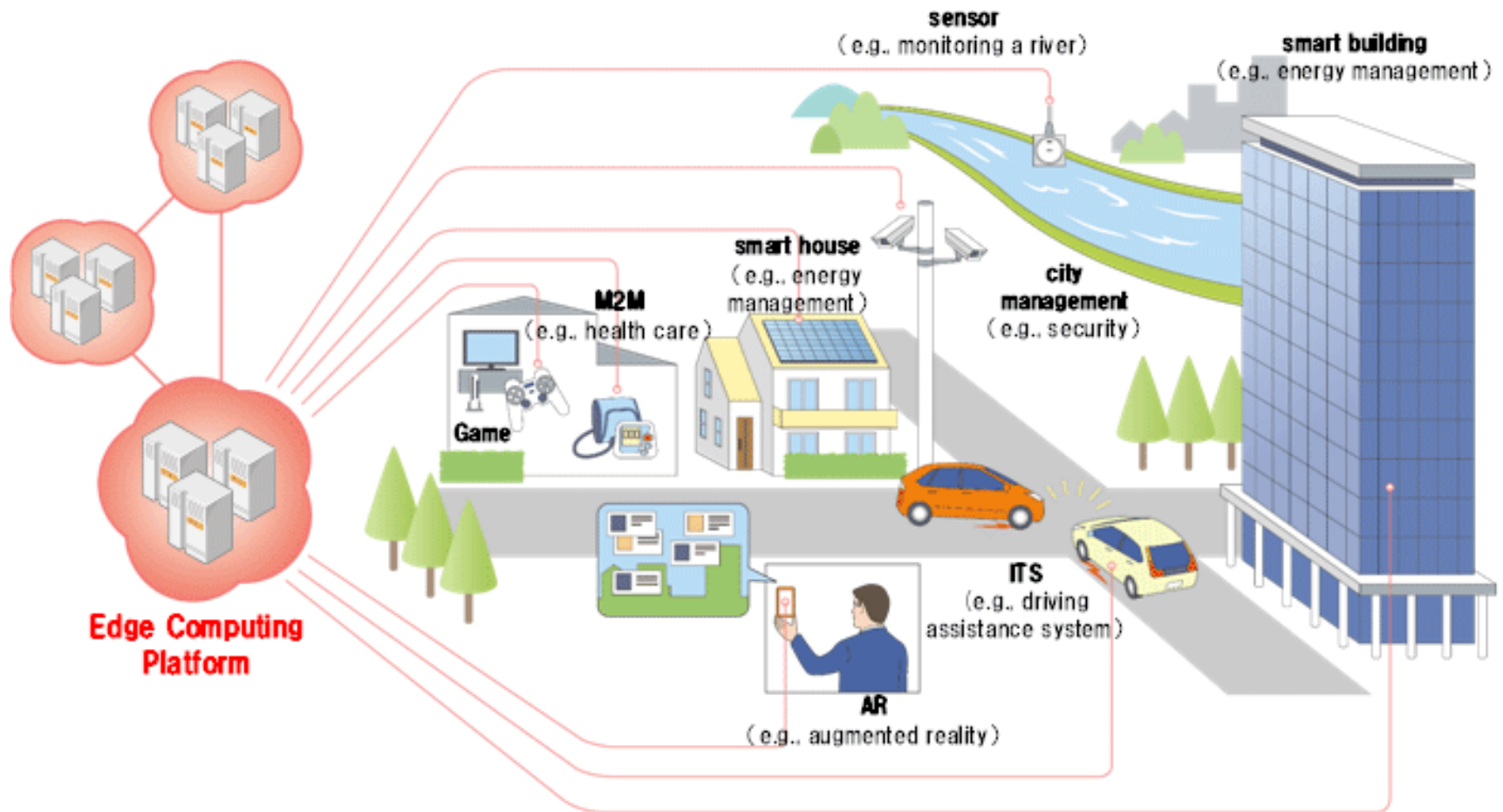
The Edge



Edge $\leftrightarrow$ Fog $\leftrightarrow$ Cloud

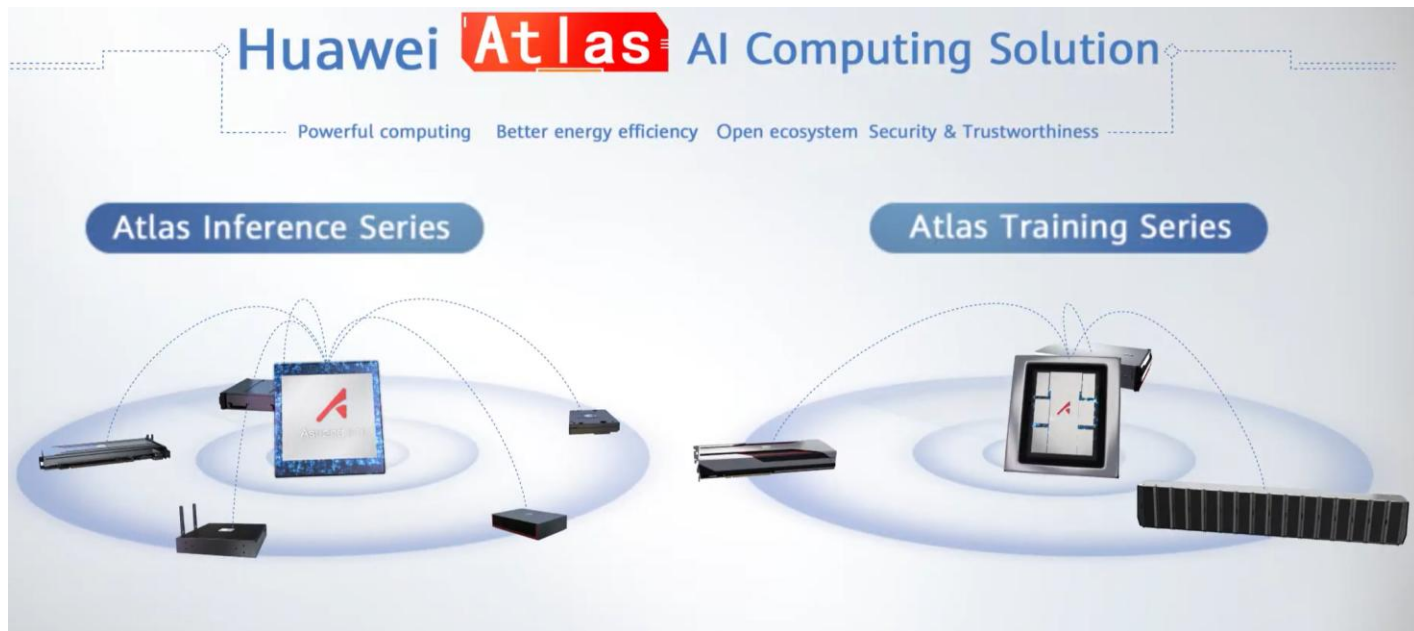


AI+Fog Computing → opportunities



Edge-Cloud use case Huawei Atlas AI Computing Solution

<https://www.youtube.com/watch?v=K-7xKcwIGVg>



A digital word: challenges for environment and industries

Players are facing huge challenges but across the globe relevant initiatives are present

- Internet of Things (IoT)
- Industry 4.0 (now 5.0)
- Artificial intelligence
 - increasing relevance in every application!

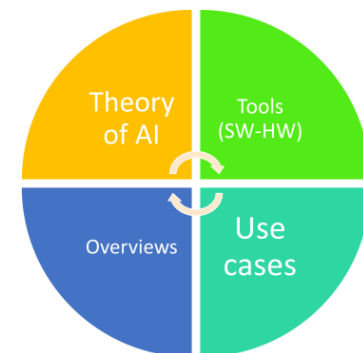




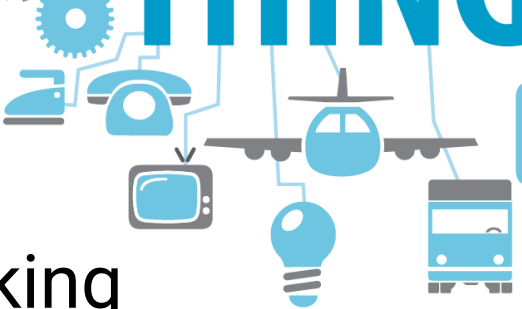
THEORY

Internet of Things

Computation of (small) devices



Internet of things (IoT)

- Since 2000, when IPv6 was first introduced, and 2007, when the 6LoWPAN wireless networking extension was released, virtually all IP-connected devices, wired and wireless, can be connected.
 - Such networks, known as the “Internet of Things,” is generating huge interest among developers of industrial-control networks.
- 
- An illustration in the top right corner depicting the Internet of Things (IoT). It features a central blue router with several lines radiating from it to connect various devices. These devices include a blue gear, a blue steam locomotive, a blue television set, a blue lightbulb, a blue airplane, and a blue train. The background is white, and the text is in a bold, black, sans-serif font.



IoT

Past ideas and Myths

- The alarm clock wake you a little bit before for the increasing traffic..
- The fridge is asking for more milk
- ...



Present and Future

- Self checking devices
- Autoconfigurations and updates
- Personal health sensors connected to the hospitals
- Cars communicating each other

Three Types of IoT Data Sources

- **Passive data**
- **Active data**
- **Dynamic data**



IoT **Passive** Data Sources



- Is the ability to consume data from sensors that don't actively communicate
 - These are sensors that must be activated before they can transmit data, and they only produce data when asked to do so
 - For example, a sensor that measures ground water saturation only produces current data when the API is invoked
- Passive data does not mean a passive data/IoT application
 - Indeed, since the sensors need to be managed, the application must take on the logic needed to do so
 - These are typically sensors that are low power and exist in remote locations

IoT **Active** Data Sources



- It means that the sensor is typically **streaming data**, such as from a jet engine
 - Different from passive data sources where you have to ask constantly for data
- → **Need to absorb the data in near real-time**, and the applications need sophisticated IoT data communications capabilities
- **Data can't be lost**, but it must be correctly parsed from the stream, and placed in the correct format for both storage and processing

IoT **Dynamic** Data Sources



- The most sophisticated and the most useful
- These are devices with sensors that communicate dynamically (bidirectional) with IoT applications, such as a smart thermostat
 - These types of sensors carry out a “conversation” with IoT applications
 - Providing a full range of capabilities:
 - Ability to change the data that's produced
 - Change the **format** of the data
 - Change the **frequency**
 - Deal with **security** issues
 - Automated **software updates** to dynamically deal with issues/versioning/calibration

IoT **Dynamic** Data Sources



- Data from dynamic data source is like an IoT application **talking** to another application.
- They do not just deal with data points that devices produce but **can alter the data** produced to meet the needs of the target IoT application.
- These are also self- and auto-configurable.
- These dynamic capabilities really **provide the best IoT capabilities**.
- **Here we can implement AI systems!**

Nest = Simple thermostat + AI → Google bought the Nest company for \$3.2 billion in 2014

3 categories IoT based on the usage/client

- **Consumer IoT** includes the connected devices such as smart cars, phones, watches, laptops, connected appliances, and entertainment systems.
- **Commercial IoT** includes things like inventory controls, device trackers, and connected medical devices.
- **Industrial IoT** covers such things as connected electric meters, waste water systems, flow gauges pipeline monitors, manufacturing robots, and other types of connected industrial devices and systems.



IoT: 3 main protocols

Different
Features
about

- **Realtime**
- **Deterministic**
- **Human inter.**
- **Security**



In a **deterministic** system, every action, or cause, produces a reaction, or effect,

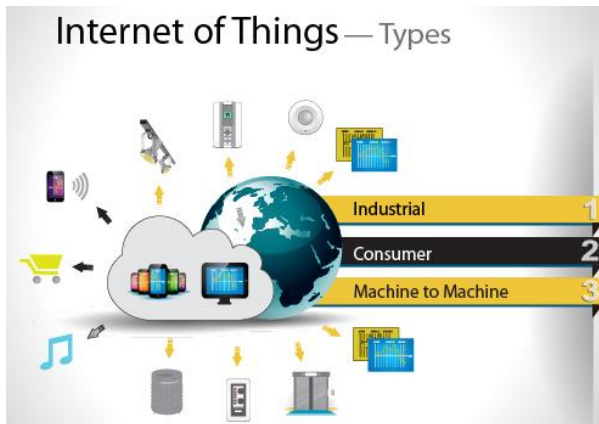
Consumer IoT

Examples

- Viewing a video on a cell phone,
- Starting up an exercise monitor to send your statistics to your account in the cloud,

Features

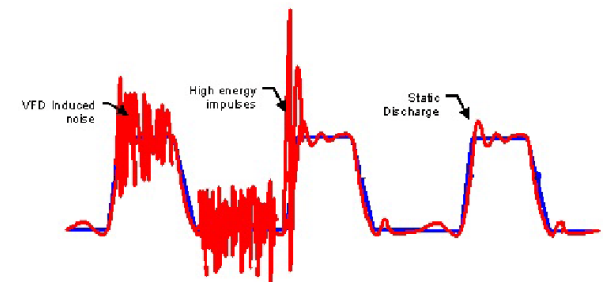
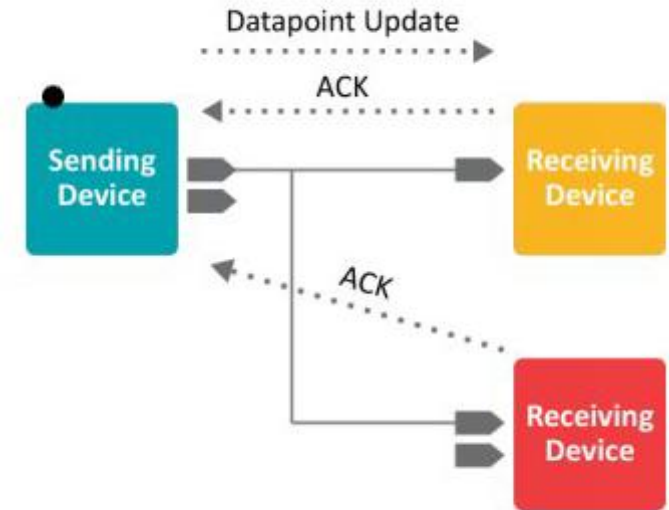
- non-real-time
- non-deterministic
- a human interacting with a device
- In case of failure, a human can recover or restart the application
- In the consumer IoT, communications run between client/server and are often streaming large amounts of data



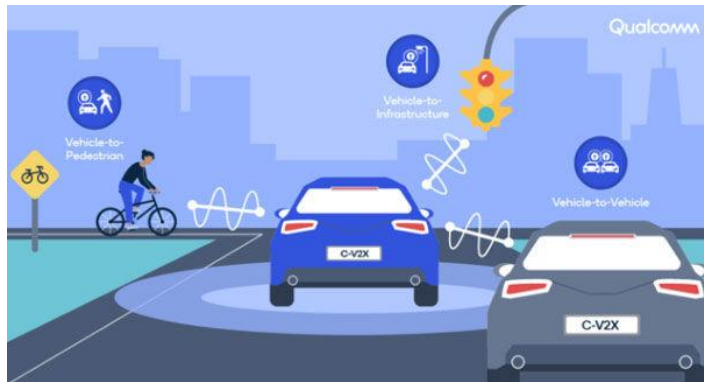
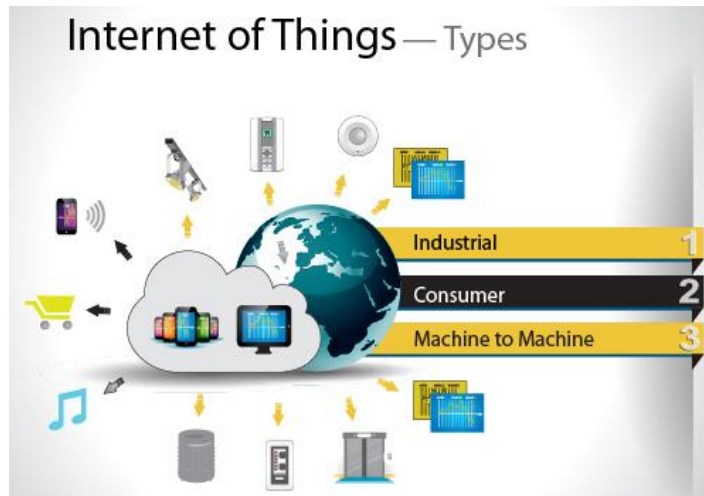
Industrial Internet of things (IIoT)

Features

- Resilience to failures
 - Packet recovery
 - Real-time requirements
 - Reliable network-wide delivery
- Security
- Physical connectivity requirements (a lot of electromagnetic interf.)
- Control services



Machine to Machine IoT



Example: Vehicle to everything applications (V2X) presented in the next lesson

Typical of client/server-based application-monitoring architecture of M2M are vehicle-tracking systems, systems that monitor a building's mechanisms for signs of wear, or systems that track mobile hospital equipment

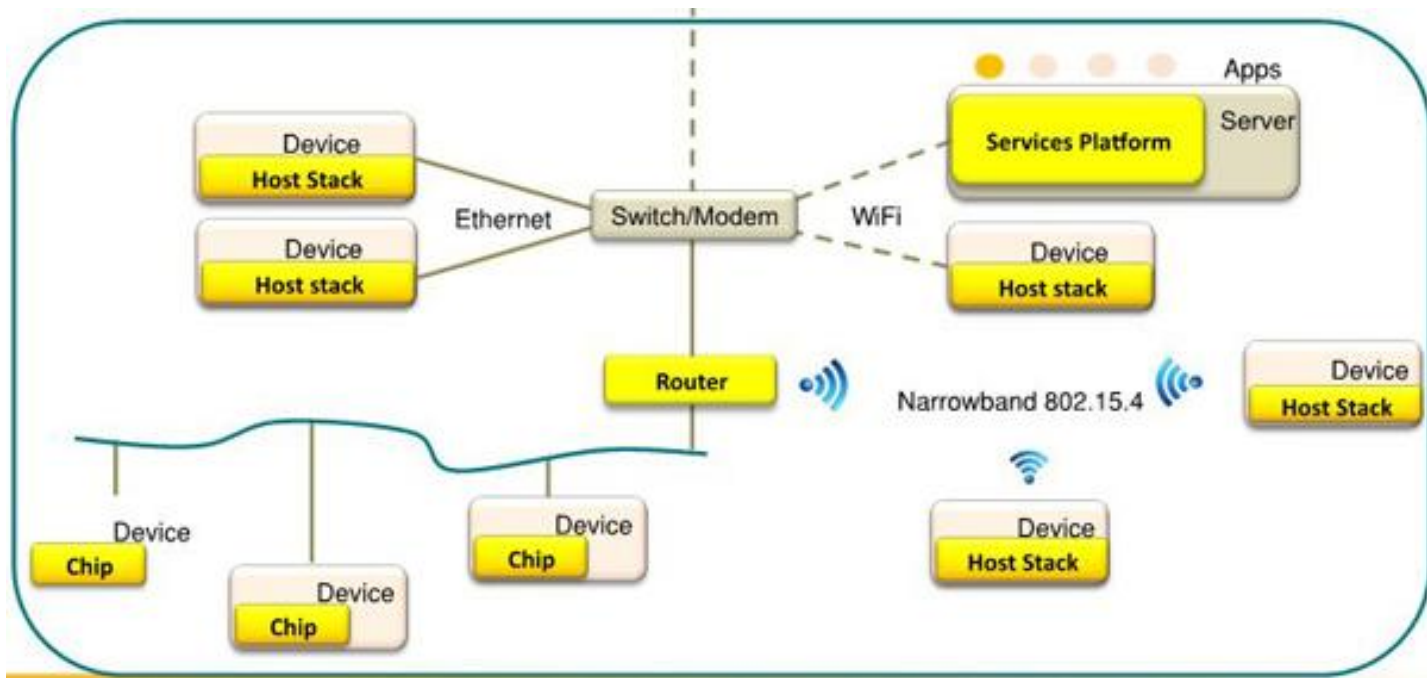
It uses client/server communications and sends smaller amounts of data (device identifier, position coordinates, and a time stamp).

What is important:

- the **reliability of communications** (no human operator or user to aid in recovery from error).
- The items the **data locates are valuable**, as well as the knowledge of where they are at any given point. Cost is incurred when the information is not available or is unreliable

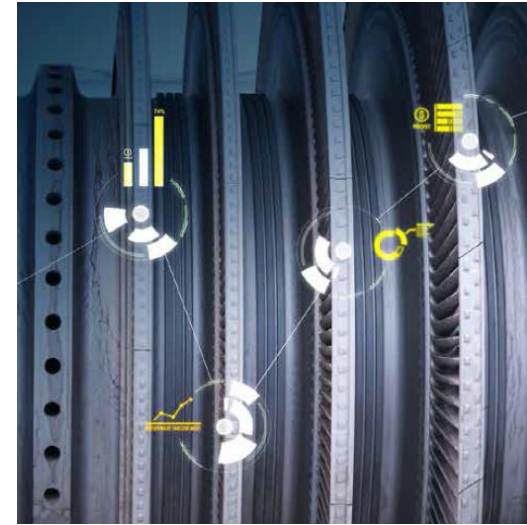
Industrial Internet of Things: typical structure of nodes

- Characterized by many-to-many connections where groups of nodes work together on a single task.



Industrial Internet of things: typical applications

- According to Oxford Economics IIoT is one of the future major trends with significant implications for the global economy impacting
 - manufacturing,
 - mining,
 - agriculture,
 - oil and gas,
 - utilities.
- Durable physical goods, general to conduct business, such as organizations that operate hospitals, warehouses and ports or that offer transportation, **logistics** and **healthcare** services.



**Sensors embedded
in objects and
machinery with
wireless connection**

A global impact... the smart city



Next step:

The Internet of Everything:

Networked Connection of People, Process, Data, Things



People

Connecting people
in more relevant,
valuable ways



Process

Delivering the right
information to the
right person (or
machine) at the
right time



Data

Leveraging data
into more useful
information for
decision making



Things

Physical devices
and objects
connected to the
Internet and each
other for intelligent
decision making

The seamless interconnection and autonomous coordination of massive number of computing elements and sensors, inanimate and living entities, people, processes and data through the Internet infrastructure

Main points



- Design of applications with ML (very important topic)
 - The 9 rules of the design
 - The ML workflow
 - Selection of the best ML technique
- Edge $\leftrightarrow$ Fog $\leftrightarrow$ Cloud
- The Internet of Things (IoT)
- Application of intelligent systems to IoT
 - Smart cities
 - Smart transportation

